

Matrix Verification Challenge

MGP 2025 Spring

Due: 13 APR (SUN) 23:59

Hello, MGP class!

We're about to dive into coding two essential routines in linear algebra: *GEMV* and *GEMM*. Here's what that means and what we're setting out to do:

- ***GEMV* (General Matrix-Vector Multiplication):** This routine multiplies a matrix by a vector.
- ***GEMM* (General Matrix Multiplication):** This routine multiplies two matrices.

The Verification Challenge:

Suppose we have three $N \times N$ matrices A , B , and C . We want to verify that A multiplied by B indeed equals C . Two different approaches can be employed to achieve this:

1. Parallel *GEMM* Approach:

By implementing *GEMM* in a parallelized manner—leveraging multi-threading, GPUs, or other parallel architectures—we can quickly compute the full matrix product.

This direct method confirms the correctness of the multiplication by comparing the computed result with C .

2. Freivalds' Algorithm:

This is a probabilistic verification technique that uses *GEMV* to check the result with high confidence. The key idea is that if $A \times B$ equals C , then for any randomly chosen N -dimensional vector v , it must be true that:

$$A \times (B \times v) = C \times v$$

By selecting a random vector v and comparing $A \times (B \times v)$ with $C \times v$, we can efficiently detect errors in the multiplication without having to compute the entire product deterministically. More details on this algorithm can be found on [Freivalds' algorithm](#).

Our Plan:

We'll implement both the parallel *GEMM* and Freivalds' algorithm. Once both are working,

we can compare their performance to determine which method is faster and more efficient under various conditions.

Let's get started on this exciting project and see which approach comes out on top!

[Overview]

Your task is to implement efficient, parallelized versions of *GEMV* and *GEMM* in the `hw2/parallel.h` header file. You should leverage parallel programming techniques to distribute the work among multiple threads and improve performance.

Additionally, you're free to design an `init_vec` function that suits your approach for initializing vectors—this might involve setting up vectors with specific values or preparing them in a way that maximizes the benefits of parallel execution.

The main objectives are to ensure correctness of the operations while achieving significant speedup compared to serial implementations. Please note that you can access the example data at `/data/hw2`, and it'll be changed while grading. (Size won't be changed.)

You can use small data at `/data/hw2/debug` for debugging purposes, but keep in mind that grading will not consider debug data at all.

[Code - 10 pts]

The total is 10 pts.

- 2 pts, if your both programs work. No partial point.
- 4 pts, if your parallel *GEMM* approach runs within **10.0 sec**.
- 4 pts, if your parallel Freivalds' algorithm runs within **0.05 sec**.

Bonus points are awarded if your program outperforms and is not a late submission.

- 1 pt, if your parallel *GEMM* approach runs faster than **4.2 sec**.
- 1 pt, if your parallel Freivalds' algorithm runs faster than **0.0015 sec**.
- 1 pt, if you are the best. (*GEMM*/*GEMV* respectively)

[Report - 10 pts]

You have to submit a simple report about your homework, including

1. How did you implement your parallel algorithm

2. Which approach is better
3. Reason Why Freivalds' algorithm is probabilistic and its error bound
4. Reasons why your program is performing at its best and proving whether performance margin is there or not
5. Evaluation including tables or graphs
6. Something else

[Note]

Please note that I will copy only `hw2/parallel.h` for grading. Your code's speed is measured based on the maximum of 5 executions on the grading server. I won't regrade due to "It worked on my machine" excuses, so I encourage you to perform the assignment more generously than the criteria. As many students tend to rush near the deadline, your code may appear slower in the local environment. Therefore, I recommend completing the assignment **as early as possible** when the server is less congested.

[Rules]

1. Put your HW2 directory under `<your_home>/mgp/hw2`
 - a. All your make commands, especially 'make run' should work.
2. **Do not change any** of the output(i.e. 'std::cout') statements. We will grade based on the program outputs. Any change on the outputs would probably cause an error, leading to 0 points for your HW assignment.
3. Don't touch anything in the directory after the deadline. If any of the timestamps are later than the deadline, we will treat it as 'late submission'.
4. You can't use OpenMP for this assignment.
5. Please retain adequate permission on your files to avoid plagiarism.
6. No abusing. Please use your common sense.
7. Zero GPT policy. No mercy.
8. PLEASE 本當に *제발* read thoroughly this document.